

PRESIDENCY UNIVERSITY

CSE324: Database Management Systems Laboratory

Class 01: Introduction to Database Systems & Relational Model

From Raw Data to an Actual Relational Database

Afzal Hossan
Lecturer
Department of Computer Science and Engineering
Presidency University

Today's Learning Path



Lecture philosophy

This is a **laboratory class**. We use ready-made SQL only to **observe database concepts**. Formal SQL syntax is taught later in Lectures 3–4.

Learning Outcomes

By the end of this class, you should be able to:

- distinguish **data**, **information**, **database**, **DBMS**, and **database system**;
- explain why structured data is better managed through a DBMS than isolated files;
- recognize the basic DBMS environment: server, client, connection, schema/database, table;
- explain relation, tuple, attribute, domain, schema, instance, degree and cardinality;
- distinguish super, candidate, primary, alternate, composite and foreign keys;
- explain entity integrity and referential integrity;
- connect to MySQL using MySQL Workbench;
- create and inspect a small relational database;
- interpret common constraint failures;
- submit a clean executable `L01_university.sql` file.

Course-plan alignment: CO1 + CO2

Part I

Why do we need a database?

Opening Problem: One Giant Spreadsheet?

Suppose a university stores everything in one sheet:

Student ID	Student Name	Department	Dept. Office	Course
231001	Rahim	CSE	Room 501	CSE324
231002	Karim	CSE	Room 501	CSE323
231003	Nabila	EEE	Room 402	EEE301
231001	Rahim	CSE	Room 501	CSE323

Think: What can go wrong when this file becomes **large**, **shared**, and **frequently updated**?

- repeated information; inconsistent copies; difficult updates
- weak relationship control; difficult security; accidental loss/corruption

Data vs Information

Data

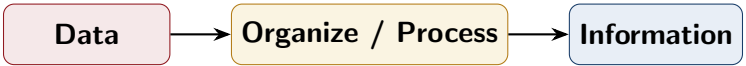
Raw facts or recorded values.

```
231001
Rahim Ahmed
CSE
3.00
```

Information

Data organized in a meaningful context.

Student 231001 is Rahim Ahmed from the CSE department.



Database, DBMS, and Database System

Database

An organized collection of **related data**.

Example: students, courses, departments and registrations.

DBMS

Software that defines, stores, retrieves, modifies and controls database data.

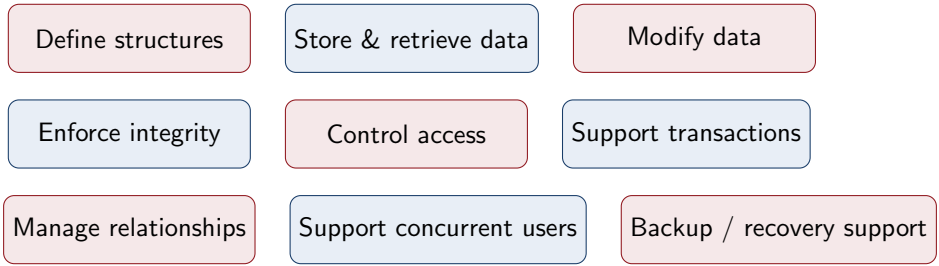
Examples: MySQL, PostgreSQL, SQL Server, Oracle.

Database System

The complete environment: **database + DBMS + users + applications + hardware + procedures**.



What Does a DBMS Actually Do?



Today

We directly experience **structure, storage, retrieval, relationships and integrity**. Transactions, security, indexing and application development come later.

File-Based Approach vs DBMS

Separate files / spreadsheets	DBMS approach
Data scattered across independent files	Integrated, managed database
High unnecessary duplication may occur	Redundancy can be controlled
Copies can become inconsistent	Better consistency mechanisms
Relationships are implicit/manual	Relationships can be explicitly enforced
Security is scattered	Centralized authorization mechanisms
Integrity depends heavily on users/programs	DBMS constraints can protect data
Concurrent editing is difficult to coordinate	Designed to support multiple users
Backup/recovery is difficult to coordinate	DBMS can provide recovery facilities
Applications can be tightly tied to file format	Better logical separation of data and programs

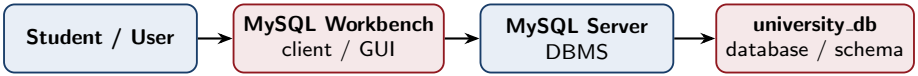
Important precision

A DBMS does **not** magically remove all redundancy. It helps us **control unnecessary redundancy** and maintain consistency.

Part II

The database environment and relational model

Our Laboratory Environment



Client

MySQL Workbench is the interface used to send commands to the server.

Server

MySQL Server is the DBMS service that stores and manages the database.

Connection: What Are We Connecting To?

A database connection typically needs information such as:

Hostname	**Port**	**Username**	**Password**
e.g., localhost	commonly 3306	DBMS account	authentication

Do not confuse these

MySQL Workbench is not the database. It is the client interface. **MySQL Server** manages the database.

Database, Schema, and Metadata

Schema in relational theory

A **schema** describes the logical structure of a relation/database.

```
Student(
StudentID, StudentName,
Email, DeptID)
```

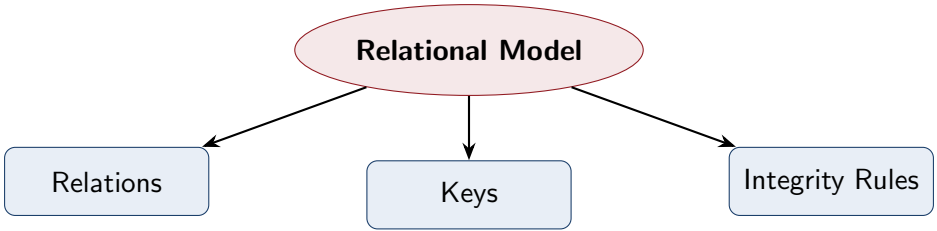
In MySQL

`DATABASE` and `SCHEMA` are effectively used as synonyms for the database container.

Metadata = data about data

Metadata describes the structure, for example:

- table names;
- column names;
- data types;
- key definitions;
- constraints.



Core idea

A relational database organizes data primarily as **relations**. In everyday DBMS use, we usually visualize a relation as a **table**.

Anatomy of a Relation

StudentID	StudentName	DeptID
231001	Rahim	CSE
231002	Nabila	EEE
231003	Karim	CSE

Relation

Conceptually, a set of tuples described by attributes.

Practical view: table

Attribute

A property describing the data.

Practical view: column

Tuple

One record in the relation.

Practical view: row

Domain

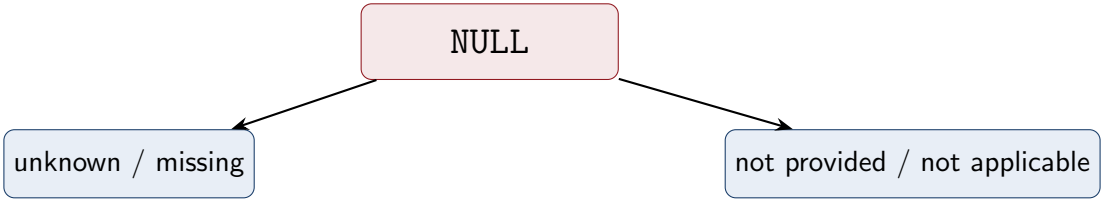
The permitted set or kind of values associated with an attribute.

Meaning	Example SQL type	Typical use
Whole number	`INT`	counts / numeric IDs
Fixed text	`CHAR(n)`	short fixed-length codes
Variable text	`VARCHAR(n)`	names / email
Exact decimal	`DECIMAL(p,s)`	money / precise values
Date	`DATE`	date of birth
Date + time	`DATETIME`	event timestamp
Logical state	`BOOLEAN`	true/false-style status

Design habit

Choose a data type according to the **meaning and operations** of the data. A phone number is not automatically an integer just because it contains digits.

What Does NULL Mean?



NULL

Absence of a known value.

0

A known numeric value: zero.

Empty string

A known text value containing zero characters.

Relation Schema vs Relation Instance

Schema = structure

```
Student(
StudentID, StudentName,
Email, DeptID)
```

Usually changes less frequently.

Instance / state = current tuples

ID	Name	Dept
231001	Rahim	CSE
231002	Nabila	EEE

Actual records can change over time.

Same structure, changing contents.

Degree and Cardinality

StudentID	Name	DeptID
S01	Rahim	CSE
S02	Karim	CSE
S03	Nabila	EEE

Degree

Number of **attributes**.

Here: **3**

Cardinality

Number of **tuples in the current relation state**.

Here: **3**

Later, "cardinality" will also appear in relationship modeling (1:1, 1:M, M:M).

Basic Properties of a Relation

- A relation has a name.
- Attributes have names and associated domains.
- Each tuple represents one record.
- Conceptually, duplicate tuples are not part of a mathematical relation.
- Tuple ordering is not conceptually significant.
- Attribute position should not define business meaning.
- Basic relational values should be atomic/single-valued.
- Keys help identify and connect tuples.

Do not jump ahead

We are **not** doing normalization today. These properties only prepare the vocabulary needed for later design work.

Part III

Keys, relationships, and integrity

Why Do We Need Keys?

Consider two students with the same name:

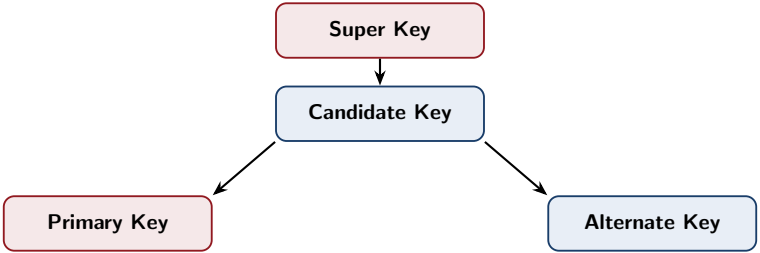
StudentID	StudentName	Email
231001	Rahim Ahmed	rahim1@pu.edu.bd
231127	Rahim Ahmed	rahim2@pu.edu.bd

Question

Can `StudentName` reliably identify a student? **No.**

A key gives the DBMS a dependable way to identify or reference a tuple.

Key Family

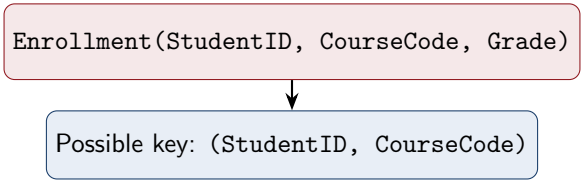


For `Student(StudentID, Email, StudentName)` if `StudentID` and `Email` are each unique:

- **Super key**: any attribute set that uniquely identifies a tuple.
- **Candidate key**: a minimal super key.
- **Primary key**: the candidate key selected as the main identifier.
- **Alternate key**: a candidate key not selected as the primary key.

Composite Key

A **composite key** contains more than one attribute.



Today

We only introduce the idea. We will not build the `Enrollment` relation yet, because relationship design and richer schema construction come later.

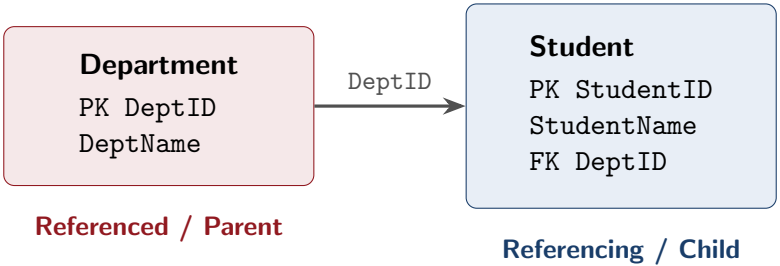
Primary Key vs Foreign Key

Primary Key	Foreign Key
Identifies a tuple in its relation	References a candidate/primary key in a related relation
Chosen to be unique	Values may repeat
Primary-key value cannot be NULL	May be NULL if the design allows it
One chosen primary key per table	A table can contain multiple foreign keys
Represents entity identity	Represents a relationship/reference

Example

`Department.DeptID` is a primary key.
`Student.DeptID` can be a foreign key referring to it.

Parent and Child Relations



Entity Integrity

Rule

The primary key must provide a valid identity for every tuple.

Valid

ID	Name
231001	Rahim
231002	Karim

Invalid idea

ID	Name
231001	Rahim
231001	Karim

Primary-key values must be **unique** and **not NULL**.

Referential Integrity

Suppose the only departments are:

`CSE` `EEE` `BBA`

Valid reference

`Student(..., DeptID='CSE')`

✓ Parent row exists.

Invalid reference

`Student(..., DeptID='LAW')`

✗ No LAW department exists.

Rule

A non-NULL foreign-key value must match an existing referenced key value.

Concept Check Before the Lab

For the relation

```
Student(StudentID, StudentName, Email, DateOfBirth, DeptID)
```

answer quickly:

1. What is the **degree**?
2. Which attribute should be the **primary key**?
3. If three rows exist, what is the **cardinality**?
4. Which attribute could connect Student to Department?
5. What integrity rule is broken by a nonexistent department code?

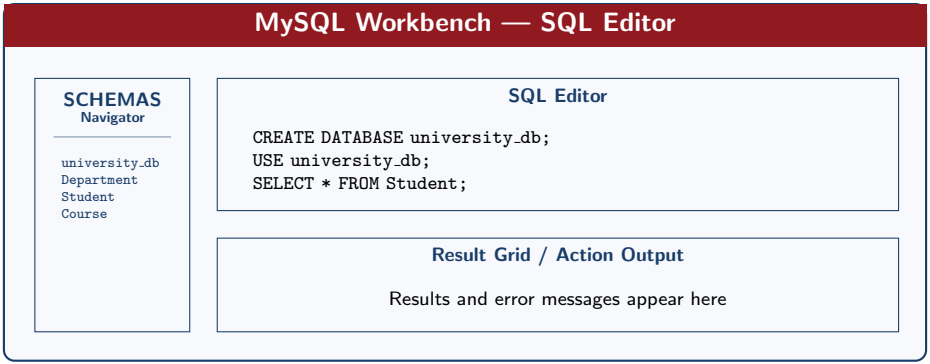
Expected

5; StudentID; 3; DeptID; referential integrity.

Part IV

Hands-on: build the first working relational database

MySQL Workbench Orientation



Today you need only: **connection**, **Schemas panel**, **SQL editor**, **Execute**, and **Result/Output**.

Step 1: Create the Database

```
CREATE DATABASE university_db;
USE university_db;
```

What happens?

MySQL creates a database container named `university_db` and makes it the active database for the current session.

Today is not the SQL lecture

You only need to understand **what the commands accomplish**. Formal DDL syntax comes in Lecture 3.

Step 2: Create the Department Relation

```
CREATE TABLE Department (
    DeptID CHAR(3) PRIMARY KEY,
    DeptName VARCHAR(100) NOT NULL
);
```

Relation	Degree	Primary key	Required
Department	2	DeptID	DeptName

Step 3: Insert and Inspect Departments

```
INSERT INTO Department (DeptID, DeptName)
VALUES
('CSE', 'Computer Science and Engineering'),
('EEE', 'Electrical and Electronic Engineering'),
('BBA', 'Business Administration');

SELECT * FROM Department;
```

Observe, do not memorize syntax yet

After insertion: **degree = 2**, **cardinality = 3**.

Step 4: Create the Student Relation

```
CREATE TABLE Student (
    StudentID VARCHAR(10) PRIMARY KEY,
    StudentName VARCHAR(100) NOT NULL,
    Email VARCHAR(100),
    DateOfBirth DATE,
    DeptID CHAR(3) NOT NULL,
    FOREIGN KEY (DeptID)
        REFERENCES Department(DeptID)
);
```

Question: Why must `Department` exist before this foreign key can reference it?

Read the Student Relation Like a Database Designer

Student

PK StudentID	VARCHAR(10)
StudentName	VARCHAR(100), NOT NULL
Email	VARCHAR(100)
DateOfBirth	DATE
FK DeptID	CHAR(3), NOT NULL

Degree	PK	FK	Parent
5	StudentID	DeptID	Department

Step 5: Insert Students

```
INSERT INTO Student
(StudentID, StudentName, Email, DateOfBirth, DeptID)
VALUES
('231001', 'Rahim Ahmed',
 'rahim@pu.edu.bd', '2004-05-10', 'CSE'),
('231002', 'Nabila Islam',
 'nabila@pu.edu.bd', '2003-11-20', 'EEE'),
('231003', 'Karim Hasan',
 'karim@pu.edu.bd', '2004-01-15', 'CSE');

SELECT * FROM Student;
```

Check

After insertion: **degree = 5**, **cardinality = 3**.

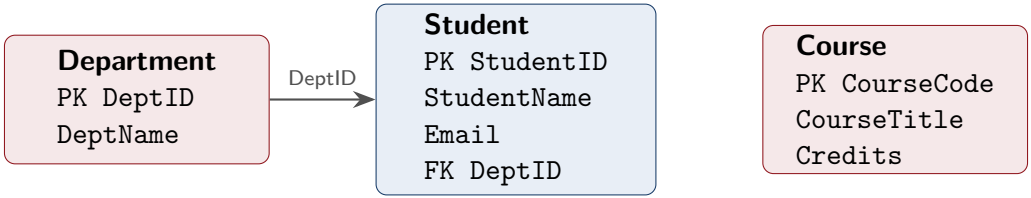
Step 6: Create and Populate Course

```
CREATE TABLE Course (
    CourseCode VARCHAR(10) PRIMARY KEY,
    CourseTitle VARCHAR(100) NOT NULL,
    Credits TINYINT NOT NULL
);

INSERT INTO Course (CourseCode, CourseTitle, Credits)
VALUES
('CSE323', 'Database Management Systems', 3),
('CSE324', 'Database Management Systems Laboratory', 1),
('CSE211', 'Data Structures and Algorithms', 3);

SELECT * FROM Course;
```

What We Have Built So Far



Notice the gap

`Course` is not yet connected to `Student`. We will later need additional relational structure to represent registration. Do not force that design today.

Part V

Make the DBMS reject bad data

Experiment 1: Duplicate Primary Key

```
INSERT INTO Student
(StudentID, StudentName, Email, DateOfBirth, DeptID)
VALUES
('231001', 'Another Student',
 'another@pu.edu.bd', '2004-02-10', 'CSE');
```

Expected

✗ The DBMS rejects the row.

Why?

231001 already identifies another tuple. The key cannot be duplicated.

Concept: primary-key uniqueness / entity integrity

Experiment 2: Invalid Foreign Key

```
INSERT INTO Student
(StudentID, StudentName, Email, DateOfBirth, DeptID)
VALUES
('231004', 'Test Student',
 'test@pu.edu.bd', '2004-03-10', 'LAW');
```

What should happen?

If `LAW` does not exist in `Department`, MySQL should reject the row because the child would reference a nonexistent parent.

Concept: referential integrity

Experiment 3: Missing Required Value

```
INSERT INTO Student
(StudentID, StudentName, Email, DateOfBirth, DeptID)
VALUES
('231005', NULL,
 'missing@pu.edu.bd', '2004-04-10', 'CSE');
```

Expected

✗ Rejected because `StudentName` was declared `NOT NULL`.

Think like a designer

Why is the student's name mandatory in our current business rule?

What Did the Three Failures Teach Us?

Invalid operation	What the DBMS protects	Concept
Duplicate `StudentID`	Unique tuple identity	Entity integrity / key uniqueness
Nonexistent `DeptID`	Valid parent–child reference	Referential integrity
`NULL` student name	Required attribute value	NOT NULL constraint

Important

Do not memorize MySQL error numbers. Be able to explain **which rule was broken and why the data should be rejected**.

Part VI

Challenge, submission, and wrap-up

End-of-Class Design Challenge

You receive this flat structure:

StudentID	StudentName	DepartmentName	CourseCode	CourseTitle
231001	Rahim	CSE	CSE324	DBMS Laboratory
231001	Rahim	CSE	CSE323	DBMS
231002	Nabila	CSE	CSE324	DBMS Laboratory

Your task

Identify the different types of things represented here and propose which data should become separate relations.

Expected recognition

At minimum: **Student**, **Department**, and **Course**. We are **not** doing formal ER modeling or normalization yet.

`L01_university.sql`

The file should contain:

- database creation and selection;
- `Department`, `Student`, `Course`;
- primary keys;
- Department–Student foreign key;
- valid sample data;
- simple result-display statements;
- clear SQL comments;
- three error experiments kept commented;
- clean, executable main script.

Keep Error Tests Commented in the Final File

```
-- ======================================================
-- ERROR EXPERIMENTS
-- Uncomment ONE test at a time.
-- ======================================================

-- Test 1: Duplicate primary key
/* INSERT INTO Student (...) VALUES (...); */

-- Test 2: Invalid foreign key
/* INSERT INTO Student (...) VALUES (..., 'LAW'); */

-- Test 3: Missing required value
/* INSERT INTO Student (...) VALUES (..., NULL, ...); */
```

Why?

The main submission remains runnable from top to bottom, while still preserving the experiments you performed in class.

Suggested 10-Mark Lab Check

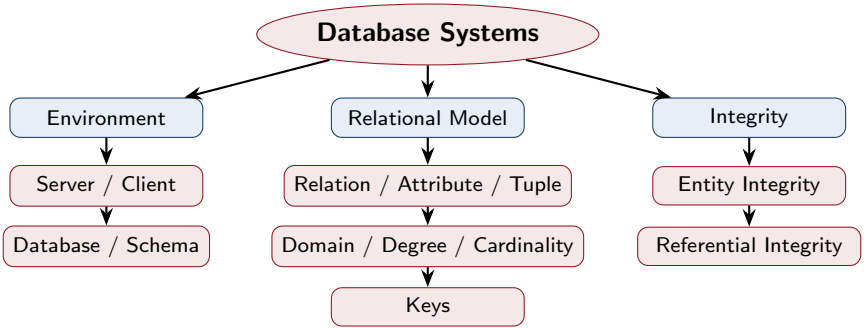
Criterion	Marks
Database created correctly	1
Department relation correct	1
Student relation correct	1
Course relation correct	1
Primary keys correct	1
Foreign key correct	1
Valid sample data inserted	1
Can explain PK/FK + relational terms	1
Can explain the three integrity failures	1
Clean executable submission	1
Total	**10**

Exit Check

Before you leave, be able to answer:

1. What is the difference between a **database** and a **DBMS**?
2. In a relation, what is a **tuple**?
3. What is the difference between **degree** and **cardinality**?
4. Why cannot two tuples share the same primary-key value?
5. Why did `DeptID='LAW'` fail?

Lecture 1 Concept Map



Scope Boundary: What Comes Later?

Covered today

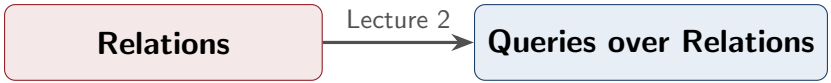
- DBMS environment
- relational terminology
- domains and basic data-type awareness
- keys and relationships
- entity/referential integrity
- first working MySQL database

Not formally taught today

- relational algebra/calculus
- full DDL/DML and querying
- joins / aggregation / subqueries
- ER/EER modeling
- normalization
- transactions/security/indexing

Next Class: Relational Algebra & Relational Calculus

Today we learned **what relations are**.



Next we ask: **How can we describe exactly which information we want from those relations?**

Thank You

Questions?

CSE324 — Database Management Systems Laboratory

Presidency University